Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

FAKE REVIEW DETECTION

USING NLP, MACHINE LEARNING AND DEEP LEARNING

The domain of the Project:

Artificial Intelligence & Machine Learning

Team Mentors (and their designation):

Mr. Gaurav Patel
Artificial Intelligence & Machine Learning Trainer

Team Members:

Mr. Supratik Mitra

Period of the project

April 2026 to June 2026

Abstract

Fake online reviews can reduce consumer trust and distort perceptions of products and services. This project presents an end-to-end Fake Review Detection system that analyzes review text and predicts whether a review is computer-generated/fake or original/human-written. The project combines natural language processing, feature engineering, multiple text representation techniques, traditional machine learning experimentation, and a two-layer LSTM deep-learning approach. A real-time application layer provides prediction and supporting text analytics, while MLflow is used for experiment tracking and Docker/Streamlit are used for reproducible deployment.

The project workflow covers data exploration, text preprocessing, feature engineering, embeddings, model development, evaluation, model serialization, and deployment. The supplied demonstration shows an English review being processed through language detection, cleaning, feature extraction, GloVe-based representation and LSTM prediction, producing a Genuine Review result with 94.2% confidence.

Keywords

Fake Review Detection • NLP • TF-IDF • GloVe • BERT • LSTM • Machine Learning • MLflow • Streamlit • Docker

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Project Architecture
- 5. Dataset and Data Processing
- 6. Text Preprocessing Pipeline
- 7. Feature Engineering
- 8. Embeddings and Text Representation
- 9. Machine Learning and Deep Learning Models
- 10. Experiment Tracking with MLflow
- 11. Deployment and Application
- 12. Project Output / Demonstration
- 13. Testing and Expected Behavior
- 14. Limitations and Future Scope
- 15. Conclusion
- 16. References

1. Introduction

The growth of e-commerce and review-driven purchasing has made online reviews an important source of information. At the same time, automatically generated or manipulated reviews can make it difficult for users and businesses to distinguish authentic feedback from artificial content. The objective of this project is to build a machine-learning-based text classification system capable of identifying review content as computer-generated/fake or original.

The implementation follows an end-to-end pipeline rather than relying only on a classifier. It includes data exploration, normalization and cleaning, linguistic feature extraction, text embeddings, model experimentation, experiment tracking, and an application interface.

2. Problem Statement

Design and implement an automated system that accepts a product review as text, processes the review using NLP techniques, represents the text numerically, and predicts whether the review is likely to be computer-generated/fake or original/human-written.

3. Objectives

• Clean and normalize raw review text.

• Handle language, spelling, numbers, punctuation and linguistic normalization.

• Extract textual, sentiment, readability and syntactic characteristics.

• Compare TF-IDF, Count Vectorization and precomputed semantic embeddings.

• Experiment with Logistic Regression, Random Forest and SVC models.

• Build and evaluate a two-layer LSTM model for sequence-based classification.

• Track parameters, metrics, datasets and artifacts using MLflow.

• Serialize the trained prediction components for application use.

• Provide an interactive Streamlit application for real-time review classification.

• Support reproducible deployment using Docker.

4. Project Architecture

The project is organized as a sequence of stages:

Project Directory Structure

```
FAKE-REVIEW-DETECTION/
├── .github/workflows/ci.yaml
├── Artifacts/
│   └── pipeline_model.joblib
├── Assets/
│   ├── frequency_dictionary_en_82_765.txt
│   └── frequency_bigramdictionary_en_243_342.txt
├── Data/
│   ├── Raw/
│   ├── Pre-processed/
│   └── Feature-Engineered/
├── Notebooks/
│   ├── N0_exploration.ipynb
```

```
│ ├── N1_Pre_processing.ipynb
│ ├── N2_Feature_engineering.ipynb
│ ├── N3_EDA.ipynb
│ ├── N4_Embeddings.ipynb
│ ├── N5_Modal_dev.ipynb
│ ├── N6_pipeline.ipynb
│ ├── mlruns/
│ └── progress_log.csv
├── Reports/
├── src/
│ ├── app.py
│ ├── model.pkl
│ ├── tfidf_vectorizer.pkl
│ └── temp.py
├── Dockerfile
├── README.md
└── requirements.txt
```

5. Dataset and Data Processing

The project uses review text together with labels indicating whether the review is computer-generated or original. The workflow creates progressively processed data, with processed outputs stored under the Data/Feature-Engineered directory.

Notebook Workflow

6. Text Preprocessing Pipeline

The inspected N6 pipeline implements a detailed text preprocessing function. It first validates the input, removes HTML tags, converts text to lowercase, converts standalone numeric values into words, removes punctuation, tokenizes the text, corrects spelling using SymSpell, optionally removes stopwords, and optionally applies stemming. Lemmatization is enabled by default and uses WordNet part-of-speech mapping for improved normalization.

1. Language detection using langdetect.
2. Optional automatic translation to English using deep-translator.
3. HTML tag removal.
4. Lowercase normalization.
5. Conversion of numbers to words using inflect.
6. Punctuation removal.
7. Word tokenization using NLTK.
8. Spelling correction using SymSpell and an English frequency dictionary.
9. Optional stopword removal.
10. Optional stemming using SnowballStemmer.
11. Default lemmatization using WordNetLemmatizer with POS mapping.

Important Implementation Note

The inspected N6 notebook contains the core preprocessing functions and imports for TF-IDF, but its single-text prediction helper functions are currently commented out. Therefore, the final application behavior should be understood from src/app.py and the serialized model/vectorizer rather than inferred solely from N6.

7. Feature Engineering

The project documentation describes feature extraction beyond simple word counts. The engineered features include lexical diversity, average word length, sentiment polarity, subjectivity, Flesch Reading Ease, sentence length and part-of-speech counts. These features are intended to capture stylistic and linguistic patterns that may differ between naturally written and generated reviews.

8. Embeddings and Text Representation

9. Machine Learning and Deep Learning Models

Traditional Machine Learning

The project experiments with Logistic Regression, Random Forest Classifier and Support Vector Classifier (SVC). Hyperparameter tuning is performed using GridSearchCV. Evaluation includes accuracy, precision, recall and F1-score, together with confusion matrices.

Deep Learning: Two-Layer LSTM

The documented deep-learning approach uses TensorFlow Keras and a two-layer LSTM architecture. Text is tokenized into sequences and padded to a common length. The model includes an embedding layer, two LSTM layers, dense layers and dropout for regularization. This sequence-based approach is intended to learn patterns in the order and context of words.

Model Selection Perspective

Using several model families allows the project to compare a simpler linear classifier, tree-based and kernel-based methods, and a sequence model. The final deployed classifier should be described according to the model actually loaded by the application. The supplied demonstration explicitly labels its model decision as an LSTM prediction.

10. Experiment Tracking with MLflow

MLflow is used to make model experimentation more systematic. Each experiment can record the input dataset/file, model type, hyperparameters, embedding type and evaluation metrics. Confusion matrices and model artifacts can also be stored. The project maintains a progress_log.csv so completed experiments do not need to be unnecessarily repeated.

11. Deployment and Application

The repository includes a Streamlit application under src/app.py and serialized prediction components including model.pkl and tfidf_vectorizer.pkl. The repository also contains an Artifacts/pipeline_model.joblib file. The README describes the project as deployed through Streamlit and provides Docker support for containerized execution.

Technology Stack

12. Project Output / Demonstration

The supplied project demonstration illustrates the complete real-time flow from a user's raw review to the final classification. The example input is an English review praising a phone. The demonstration

shows language detection, skipped translation because the text is already English, text cleaning, feature extraction, GloVe vector representation and LSTM model prediction.

Figure 1. End-to-end prediction demonstration supplied with the project.

Observed Output

The result panel also provides an explanatory model-decision message indicating coherent sentiment, natural phrasing and no detected fake-review indicators. This demonstrates the project's focus on making the prediction understandable to the end user rather than returning only a class label.

13. Testing and Expected Behavior

Representative functional tests for the application include:

14. Limitations and Future Scope

Limitations

• A text classifier can learn patterns from the training data that do not generalize to every platform or product category.

• Generated text changes over time, so periodic retraining and monitoring may be required.

• Confidence values should be interpreted as model confidence, not absolute proof that a review is fake or genuine.

• The current documentation describes multiple representations and models; the exact production behavior should always be verified against the current app.py and serialized artifacts.

Future Scope

• Fine-tune a modern transformer such as DistilBERT/RoBERTa on the project dataset.

• Combine linguistic, behavioral and metadata-based features when such data is available.

• Add explainability such as important tokens or feature contributions.

• Add batch CSV upload and downloadable prediction reports.

• Add model monitoring and drift detection.

• Provide a REST API for integration with e-commerce systems.

• Improve security, authentication and production deployment practices.

15. Conclusion

The Fake Review Detection project demonstrates a complete applied AI/ML workflow for text classification. It combines structured preprocessing, linguistic feature engineering, multiple text representations, traditional machine-learning experimentation and a two-layer LSTM approach. MLflow adds reproducibility and experiment management, while Streamlit and Docker make the trained system accessible as an application. The supplied demonstration confirms the intended end-to-end experience: a review is entered, processed, represented and classified with an interpretable result and confidence value.

16. References

• Project README.md — Fake Review Detection Project documentation supplied by the project.

• N6_pipeline.ipynb — preprocessing and single-text pipeline notebook supplied with the project.

• Scikit-learn documentation — machine learning and TF-IDF concepts.

• TensorFlow/Keras documentation — LSTM and deep-learning implementation.

• MLflow documentation — experiment tracking and artifact management.

• Streamlit documentation — interactive ML application deployment.

— End of Project Report —

Stage	Function
Input	Raw review text supplied by the user.
Preprocessing	Language handling, cleaning, tokenization, spelling correction and lemmatization/stemming.
Feature Engineering	Lexical, sentiment, readability and part-of-speech characteristics.
Representation	TF-IDF, Count Vectorization and semantic embeddings such as GloVe/BERT.
Modeling	Traditional classifiers and a two-layer LSTM.
Evaluation	Accuracy, precision, recall, F1-score and confusion matrices.
Tracking	MLflow records parameters, metrics and artifacts.
Deployment	Streamlit application with serialized model/vectorizer; Docker support.

Notebook	Role
N0_exploration.ipynb	Initial exploration and understanding of the dataset.
N1_Pre_processing.ipynb	Cleaning and normalization of textual data.
N2_Feature_engineering.ipynb	Creation of engineered textual and linguistic features.
N3_EDA.ipynb	Exploratory data analysis and visual investigation.
N4_Embeddings.ipynb	Generation/comparison of text representations and embeddings.
N5_Modal_dev.ipynb	Model development and experimentation.
N6_pipeline.ipynb	Construction of the single-text preprocessing/pipeline logic.

Representation	Purpose
TF-IDF	Represents terms according to their importance within a document relative to the corpus.
Count Vectorization	Represents text through basic term-frequency counts.
GloVe	Provides dense semantic word-vector representations; the supplied demonstration describes GloVe vec
BERT	Precomputed contextual embeddings were explored as another semantic representation.

Metric	Meaning
Accuracy	Overall fraction of correctly classified reviews.
Precision	Proportion of predicted positive/fake cases that are actually positive/fake.
Recall	Proportion of actual positive/fake cases detected by the model.
F1-score	Harmonic mean of precision and recall.

Technology	Use
Python	Core programming language.
Pandas / NumPy	Data handling and numerical processing.
NLTK / SymSpell / langdetect / inflect	Text normalization and linguistic preprocessing.
Scikit-learn	Traditional ML models and TF-IDF.

Technology	Use
TensorFlow / Keras	LSTM deep learning.
MLflow	Experiment tracking.
Streamlit	Interactive web application.
Docker	Containerized deployment.

Item	Demonstration Result
Input language	English
Word count	9 words
Sentiment	Positive
Translation	Not required
Text representation	GloVe vector representation
Model	LSTM
Prediction	Genuine Review
Confidence	94.2%

Test Case	Expected Behavior
Valid English review	The system should preprocess the text and return a classification.
Non-English review	The pipeline can detect the language and the documented design can translate to English when require
Empty input	The preprocessing function handles empty/non-string input by returning an empty/invalid representatio
Review containing numbers	Standalone digits are converted into word representations during preprocessing.
Misspelled words	SymSpell attempts spelling correction using the supplied frequency dictionary.
Punctuation/HTML	Punctuation and HTML tags are removed before token-level processing.
Model prediction	The processed representation is passed to the trained classifier used by the application.

Project Output / Demonstration

The supplied demonstration shows the end-to-end prediction interface: an English review is entered, processed through language detection, text cleaning, feature extraction and GloVe representation, and then classified using the LSTM model.

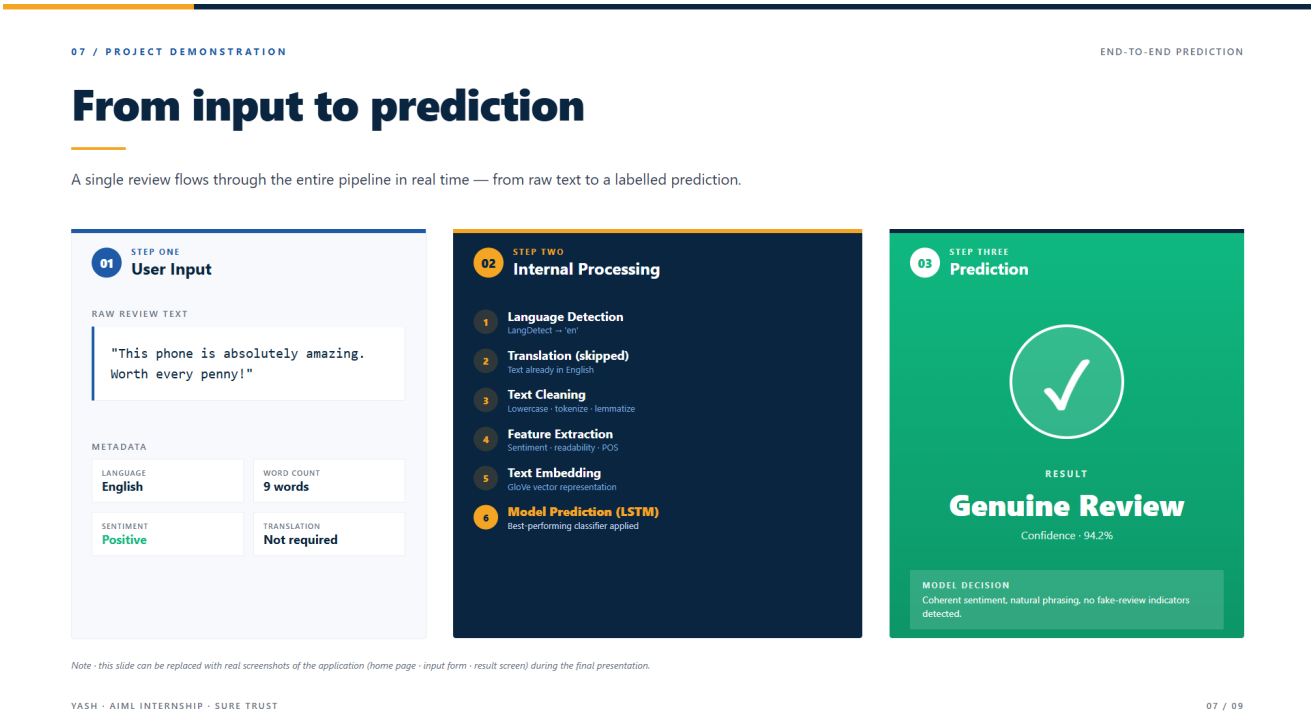

Figure: Supplied end-to-end prediction demonstration.

Observed output: Language — English; Word count — 9 words; Sentiment — Positive; Translation — Not required; Text representation — GloVe vector representation; Model — LSTM; Prediction — Genuine Review; Confidence — 94.2%.